

Day 4 Instructor Run Sheet (120 minutes)

For the instructor only. Teach in Hindi. All written material stays in English. Day 4 has two risks: the built-in SQLite module on Windows, and signing in to GitHub from Git on the lab PCs. Both are **not tested** on those PCs. Test them before class.

Driver decision

Driver decision: the lab uses the built-in `node:sqlite`.

- On Node.js 24.21.0 on Linux it ran **without a flag** and printed **no warning**. The data layer and the server were tested end to end there: add, delete, wrong password, and notes surviving a restart.
- It has **NOT been tested on Windows**.
- I could **not confirm its stability label** in the Node.js documentation.
- **If the Windows pilot fails:** the alternative is the `better-sqlite3` package (`npm install better-sqlite3`). Its API is similar but **must be checked**, and `data.js` would need to change. **Not tested**.
- Write your decision in Pilot C3 of `../INSTRUCTOR-PILOT-CHECKLIST.md` before class.

Before the class (do these first)

Do **Pilot C** (SQLite) from `INSTRUCTOR-PILOT-CHECKLIST.md` on a Windows 11 Home lab PC with Node.js 24 installed: run `node -e "require('node:sqlite')"`, then run the finished `done/notes-board` (`npm install`, create `.env`, `node server.js`, add a note, restart). Write down the result.

Do **Pilot D1** (GitHub) on a lab PC: `git init`, `git add .`, `git commit`, `git push` to a throwaway repository. Note whether a browser window opens for sign in, and whether the lab network allows it. **Not tested yet**.

Ask students (before class, if you can) to check that they can sign in at `github.com`. Students without an account need extra time, and email confirmation can delay them.

Put the Day 4 `starter` and `done` folders on the LAN share and on the spare USB sticks. Check that the lab network can reach `registry.npmjs.org` (for `npm install` of Day 3 packages, and `pg` for Track B).

Prepare **your own** test repository on GitHub, and keep your browser signed in to it, so that you can demonstrate Part 7 on the projector.

Track B only: install PostgreSQL once on a lab PC and write down the real installer screens. The lab sheet describes them only in a general way and they are **not verified**.

Zoom VS Code and the terminal (**Ctrl and +**) so that the back row can read.

Timeline

Minutes	What	Notes
0 to 8	Lab Part 0: check the setup and the starter	Walk the room. Everyone runs the Day 3 server once. Anyone without <code>notes-board</code> copies the starter now
8 to 18	Talk: database, tables, rows, columns, SQL, <code>?</code> placeholders	Draw a table on the board. Show the SQL injection idea with the unsafe string and the safe <code>?</code> version. Do not run any harmful SQL
18 to 32	Lab Part 1: first look at SQLite with <code>try.js</code>	Students type <code>try.js</code> , run it twice, see the rows grow, then delete both files. Ask the injection question at the end
32 to 52	Lab Part 2: write <code>data.js</code>	Three sections, explain each before students type it. Remind: same function names, new <code>init()</code>
52 to 58	Lab Part 3: change <code>server.js</code>	Three small edits. Show <code>init().then(...)</code> and say "prepare the database first"
58 to 68	Lab Part 4: test and restart	Gate: most students must see their note survive the restart. Pods of six: the first student who passes helps the others
68 to 76	Talk: version control, Git, <code>.gitignore</code> , why <code>.env</code> must not be committed	Use the "final2, final-REAL" example
76 to 88	Lab Part 5: <code>git init</code> , config, status, add, commit, log	Everyone reads their own <code>git status</code> and confirms that <code>.env</code> , <code>node_modules</code> and <code>notes.sqlite</code> are not listed before <code>git add</code> .
88 to 108	Lab Parts 6 and 7: GitHub repository, push, and the <code>.env</code> check	Demonstrate on the projector. Sign in is the risk. Handle failures one by one with the Stuck Sheet (Problems 18 to 24). Do Part 7 together, every student looks at their own page
108 to 115	Track B for fast finishers, quiz for the others	Track B students work on their own with the lab sheet. Everyone else does the quiz
115 to 120	Homework, backup	Remind: bring your GitHub login on Day 5. Copy the <code>Workshop</code> folder to the backup place

Cut lines (if time runs short)

Cut in this order:

1. **Track B (Part 8).** It is optional. Day 5 works without it. Skip it, or offer it to fast finishers as take-home work.
2. **The GitHub push (Parts 6 and 7) can move to the start of Day 5.** Students must still finish Parts 1 to 5 (the database and the first commit), because the push needs the commit. If you move it, say so clearly, and give the first 20 minutes of Day 5 to it.
3. **Quiz:** move it to the start of Day 5.

Do not cut: the database change (Parts 2 to 4, with the restart test) and the `git status` check in Part 5.

If sign in to GitHub fails for some students

- Use the Stuck Sheet, Problem 22. **This has not been tested on the lab PCs.**
- Help each student one at a time. Do not stop the class for one student.
- A **personal access token** is an alternative that you can explain. It is **not tested** for this workshop. Do not write tokens on the board, and tell students never to put a token in a file in their project.
- If the lab network blocks GitHub, run the push as a demo on your own computer, and let the students push on Day 5 from the network that works.
- Make a list of students who did not push. Check them first on Day 5.

What to say

- "A database keeps data on the disk. Memory is cleared when the program stops."
- "Never join user text into SQL. Use `?`. The text and the SQL travel separately."
- "SQLite is one file. Show me `notes.sqlite` in your folder."
- "`git status` before `git add`. Read it every time."
- "If you committed `.env`, tell me. Changing the password is the fix. We cannot promise to hide it again."
- "Git is on your computer. GitHub is a website that keeps a copy."

Talk points for Part A (database)

- Table, row, column, primary key. Draw the `notes` table.
- SQL words: `CREATE TABLE`, `INSERT`, `SELECT`, `WHERE`, `ORDER BY`, `DELETE`. Warn: `DELETE` without `WHERE` removes every row.
- `?` placeholders and SQL injection. It is the same idea as XSS on Day 2: text from a user was treated as code.
- `init()` runs before `app.listen`. The 3 starter notes are added only when the table is empty. Deleting all notes and restarting brings them back. This is by design.

- The database file name has no folder, so the file is made in the folder where `node server.js` is run. Many "my notes vanished" problems are this.

Talk points for Part B (Git and GitHub)

- Version control: history, go back, work together.
- `.gitignore` lists `.env`, `node_modules/` and `*.sqlite`. Show `git status` and `git status --ignored`.
- `git config user.name` is set **without** `--global` in the lab, so that the setting stays in the project (the lab PCs may be shared; how they are shared is not confirmed).
- Create the GitHub repository **empty** (no README). This avoids a rejected push (Stuck Sheet, Problem 21).
- Branch name: everyone runs `git branch -M main` before the first push. The Git version in the lab may use `master` as the default name (not confirmed on Windows).

Before ending the class

Notes stay after a restart, for everyone who got that far.

Students with a pushed repository have checked that `.env` is **not** on GitHub.

Students with problems are on your list (especially sign in and push).

Students know their GitHub login is needed on Day 5.

Backup done.

Notes for your own preparation tonight (Day 5)

- The Day 5 starter folder must be a copy of the Day 4 `done` folder.
- Render's documentation (as read for this workshop) says the filesystem of a Free web service is **not kept** on restart, redeploy or spin-down. If that holds, a SQLite file will be reset there. Test this in Pilot D6 and decide how to explain it. **Not tested.**
- Decide how to handle students who did not manage to push on Day 4.